GUÍA MAESTRA DE SUSTENTACIÓN INDIVIDUAL

Plataforma Web de Tutorías Académicas (MyT)

Documento Técnico Integral: Explicación de Arquitectura, Funcionamiento y Catálogo Exhaustivo de Evidencias de Código (Angular 22 + Spring Boot 4 + PostgreSQL)

PROYECTO
MyT (Make Your Tutor)

EVALUACIÓN
Sustentación Oral y Demostración Individual

STACK FRONTEND
Angular 22 (Standalone, Signals, Reactive Forms, RxJS)

STACK BACKEND
Spring Boot 4 / Java 21 / Spring Data JPA / REST

BASE DE DATOS
PostgreSQL 17 (Esquema app, Migraciones versionadas)

SEGURIDAD
JWT en Cookies HttpOnly + SameSite=Strict + CORS

1. Descripción del Proyecto y Funcionamiento General

1.1 ¿Qué es MyT?

MyT (Make Your Tutor) es una plataforma web integral diseñada para la gestión, reserva y administración de tutorías académicas universitarias. Permite a los estudiantes encontrar apoyo pedagógico personalizado en distintas áreas (matemáticas, programación, ciencias, idiomas, diseño), agendar sesiones según la disponibilidad horaria de los tutores, dejar reseñas con calificaciones reales y comunicarse en tiempo real.

1.2 Arquitectura del Sistema

El sistema se implementó siguiendo una arquitectura desacoplada en tres capas de alta cohesión:

- **Capa Frontend (Angular 22 SPA):** Diseñada con componentes Standalone modernos, control reactivo mediante *Signals* (sin sobrecarga de Zone.js), formularios reactivos fuertemente tipados con validaciones sincrónicas, y comunicación padre-hijo basada en las nuevas funciones `input()` y `output()`. Toda comunicación con el servidor está centralizada en `ApiService`, con interceptores que añaden cabeceras y controlan credenciales.
- **Capa Backend (Spring Boot 4 / Java 21):** API RESTful desacoplada compuesta por controladores anotados con `@RestController`, respuestas estructuradas con `ResponseEntity` (códigos 200, 201, 204, 400, 404, 409), manejo centralizado de excepciones con `@RestControllerAdvice`, y persistencia transaccional con control de concurrencia mediante bloqueo pesimista (`PESSIMISTIC_WRITE`) para garantizar que dos estudiantes no puedan reservar simultáneamente el mismo bloque horario de un tutor.
- **Capa de Persistencia (PostgreSQL 17):** Base de datos relacional administrada mediante Spring Data JPA e Hibernate en un esquema dedicado denominado `app`. Incorpora consultas optimizadas en **JPQL** con funciones de agregación (`AVG`), paginación nativa y carga *eager* de relaciones complejas mediante `@EntityGraph` para eliminar el problema de consulta N+1.

1.3 Flujo Funcional por Roles

Rol	Responsabilidades y Funcionalidades	Vistas y Acciones en el Código
Estudiante	Explora el catálogo paginado de tutores, filtra por materia o tarifa máxima, consulta el perfil detallado del tutor con sus calificaciones reales, solicita reservas seleccionando fechas y horarios disponibles de la agenda del tutor, y cancela sus tutorías con motivo justificado.	`TutoresPageComponent` (explorar), `ReservasPageComponent` (reservar), `ReservaDetallePageComponent` .
Tutor	Configura su tarifa por hora, edita su biografía académica, habilita o deshabilita su disponibilidad para reservas, define su agenda semanal y gestiona sus tutorías agendadas (confirmándolas o cancelándolas).	`TutorEditorComponent` (edición reactiva), `ReservasPageComponent` (cambio de estado), `AgendaEditorComponent` .
Administrador	Supervisa a todos los usuarios, añade el perfil tutor a cuentas de estudiantes existentes (asignando materias y tarifa inicial), desactiva tutores de forma lógica conservando historial, y visualiza la totalidad de reservas del sistema.	`UsuariosPageComponent` , Formulario rápido en `TutoresPageComponent` , Endpoints administrativos con control RBAC.

2. Matriz General de Trazabilidad de Requisitos

Esta tabla consolida los 28 elementos evaluables exigidos por la rúbrica, mapeando cada concepto con su archivo exacto, líneas de código y evidencia concreta para la sustentación.

#	Concepto Solicitado	Ubicación Exacta en el Código	Evidencia / Función en MyT
1	**Componentes Angular**	`frontend/src/app/pages/` y `shared/components/`	Múltiples componentes Standalone desacoplados (páginas, átomos, moléculas y organismos).
2	**Routing (Mínimo 2)**	`frontend/src/app/app.routes.ts:4-123`	Navegación completa: `/tutores`, `/reservas`, `/login`, `/usuarios` con `sessionGuard`.
3	**Interpolación {{ }} (Mín. 3)**	`tutores-page.html:163, 199, 223, 228`	Renderizado dinámico de `{{ p.totalElements }}`, `{{ t.nombre }}`, `{{ t.biografia }}`.
4	**Property Binding [ ] (Mín. 3)**	`tutores-page.html:46, 54, 79, 187`	Enlace de propiedades: `[disabled]="cargando()"`, `[tutor]="tutor"`, `[class.active]`.
5	**Event Binding ( ) (Mín. 3)**	`tutores-page.html:12, 55, 81, 175`	Captura de eventos: `(ngSubmit)="filtrar()"`, `(click)="alternarOrden()"`, `(guardado)="guardar($event)"`.
6	**Two-way Binding [(ngModel)] (Mín. 2)**	`tutores-page.html:18, 26, 43` y `login-form.ts:15, 24`	Sincronización bidireccional en búsqueda (`busquedaTexto`), `materiaId`, `tarifaMax` y login.
7	**Signals (signal, set, update)**	`tutores-page.ts:52, 162, 185, 307, 332, 355`	Señal `cambios = signal(0)` modificada con `update(n ⇒ n + 1)` y mostrada en interfaz.
8	**Control de flujo (@if, @switch, @for)**	`tutores-page.html:62, 76, 156, 186`	Nativo Angular 17+: `@if(error())`, `@switch(cargando())` y `@for(t of tutoresMostrados())`.
9	**Comunicación Padre-Hijo**	Hijo: `tutor-editor.ts:55-79` Padre: `tutores-page.html:78-83` y `tutores-page.ts:293`	Padre envía objeto con `[tutor]="tutor"` (`input()`); Hijo devuelve con `output()` y `emit()`; Padre recibe con `$event`.
10	**Formularios Funcionales**	`tutor-editor.ts:7-52`, `reservas-page.html`	Edición de perfil de tutor, alta de reserva y asignación administrativa de perfil.
11	**Formularios Reactivos**	`tutor-editor.ts:6, 59-69`	Uso explícito de `ReactiveFormsModule`, `FormGroup` y `FormControl` fuertemente tipados.
12	**Validaciones (Mínimo 3)**	`tutor-editor.ts:22-42, 60-67`	`required` y `maxLength(2000)` en biografía; `required`, `min(1)` y `max(9999999999.99)` en tarifa con alertas.
13	**Pipes de Angular**	`tutores-page.html:228`, `reservas-page.html:225, 272`	Uso de `number` (formato miles), `currency` (COP), `date` ('dd MMM') y `titlecase`.
14	**Servicios Angular**	`frontend/src/app/core/services/api.service.ts`	`ApiService` centralizado con CRUD completo (consultar, crear, actualizar, eliminar).
15	**Inyección con inject()**	`api.service.ts:72`, `tutores-page.ts:40-43`	Inyección moderna con `inject(HttpClient)`,

#	Concepto Solicitado	Ubicación Exacta en el Código	Evidencia / Función en MyT
			`inject(ApiService)`, `inject(AuthService)`.
16	**HttpClient**	`app.config.ts:19` y `api.service.ts:76-136`	Peticiones asíncronas con interceptor de credenciales y cabecera `X-MYT-Request`.
17	**2 Operaciones GET con subscribe()**	`tutores-page.ts:164-184` y `281-290`	1) Listado paginado de tutores: `api.tutores()`. 2) Detalle por ID para edición: `api.tutor(id)`.
18	**2 Operaciones POST con subscribe()**	`tutores-page.ts:318-336` y `reservas-page.ts:220`	1) Asignar perfil tutor: `api.crearTutor()`. 2) Crear reserva: `api.crearReserva()`.
19	**2 Operaciones PUT con subscribe()**	`tutores-page.ts:298-311` y `reservas-page.ts:240`	1) Guardar edición de tutor: `api.editarTutor()`. 2) Cambiar estado de reserva: `api.estadoReserva()`.
20	**2 Operaciones DELETE con subscribe()**	`tutores-page.ts:347-359` y `reservas-page.ts:258`	1) Desactivar tutor: `api.desactivarTutor()`. 2) Cancelar reserva: `api.cancelarReserva()`.
21	**Manejo de Errores HTTP**	`api.service.ts:63-71` y `tutores-page.html:62-68`	Manejo de caída de red (status 0), sesión expirada (401), sin permiso (403) y conflicto de datos (409).
22	**Loading (Indicador Visual)**	`tutores-page.ts:49, 162, 179` y `tutores-page.html:156`	Señal `cargando = signal(false)` activada antes de la petición y cerrada con RxJS `finalize()`.
23	**REST API con @RestController**	`backend/ ... /api/TutorRestController.java` y `ReservaRestController.java`	Controladores Spring Boot que exponen endpoints REST (2 GET, 2 POST, 2 PUT, 2 DELETE).
24	**ResponseEntity con Códigos HTTP**	`TutorRestController.java:52, 82, 87, 106, 128`	Uso explícito de `ResponseEntity` devolviendo 200 OK, 201 Created, 204 No Content, 400 Bad Request y 404 Not Found.
25	**Paginación Funcional**	Backend: `TutorRestController.java:46-83` Frontend: `tutores-page.ts:107-127, 238-254`	Paginación real de 30 tutores (6 páginas de 5 items) con botones Primero, Anterior, Siguiente y Último.
26	**Conexión PostgreSQL**	`application.properties` y migraciones `001-004.sql`	Persistencia en PostgreSQL 17 (Supabase / Docker local) en esquema `app` con RLS y pool JDBC.
27	**JPA / Hibernate**	`backend/ ... /entities/` (`Tutor`, `Reserva`, `Usuario`)	Entidades relacionales mapeadas con `@OneToOne`, `@OneToMany`, `@ManyToMany` y `@EntityGraph`.
28	**JPQL (Mínimo 3 Consultas)**	`TutorRepository.java:49, 69` y `ReservaRepository.java:43`	1) Promedio de calificación (`AVG`). 2) Filtro dinámico con paginación. 3) Reservas de estudiante por estado.

3. Desglose Detallado de Cada Elemento de Evaluación

1. Componentes Angular (Estructura y Organización)

ANGULAR 22 / FRONTEND

¿Qué hace y qué función cumple?

Organiza la interfaz gráfica en componentes Standalone desacoplados siguiendo una arquitectura atómica. Permite reutilizar lógica de presentación y aislar responsabilidades.

¿Dónde está en el código?

`frontend/src/app/pages/tutores-page/tutores-page.ts` (Página catálogo)
`frontend/src/app/shared/components/organisms/tutor-editor/tutor-editor.ts` (Hijo editor)
`frontend/src/app/shared/components/atoms/avatar/avatar.ts` (Átomo avatar)

Cómo sustentarlo en la presentación:

"El proyecto utiliza componentes Standalone modernos sin NgModules. La interfaz está dividida en páginas principales (como `TutoresPageComponent` y `ReservasPageComponent`) y componentes de soporte reutilizables como `TutorEditorComponent` y `AvatarComponent`, lo que facilita el mantenimiento y las pruebas unitarias."

2. Routing de Angular (Navegación entre Vistas)

ANGULAR ROUTER / FRONTEND

¿Qué hace y qué función cumple?

Permite la navegación declarativa tipo Single Page Application (SPA) entre las diferentes vistas del proyecto sin recargar el navegador, protegiendo rutas sensibles mediante guardias de sesión (`sessionGuard`).

¿Dónde está en el código?

Archivo: `frontend/src/app/app.routes.ts` (líneas 4 a 123). Configurado en `app.config.ts` con `provideRouter(routes)` y renderizado en `app.html` con `<router-outlet />` .

```
// frontend/src/app/app.routes.ts (extracto)
export const routes: Routes = [
  { path: '', pathMatch: 'full', loadComponent: () => import('./pages/landing-page/landing-page').then(m =
  { path: 'tutores', loadComponent: () => import('./pages/tutores-page/tutores-page').then(m => m.TutoresP
  { path: 'tutores/:id', loadComponent: () => import('./pages/tutor-perfil-page/tutor-perfil-page').then(m
  { path: 'reservas', canActivate: [sessionGuard], loadComponent: () => import('./pages/reservas-page/rese
  { path: 'usuarios', canActivate: [sessionGuard], data: { admin: true }, loadComponent: () => import('./p
  { path: 'login', loadComponent: () => import('./pages/login-page/login-page').then(m => m.LoginPageCompo
  { path: '**', redirectTo: '' }
];
```

Pregunta del evaluador: "¿Cómo se protegen las rutas que requieren inicio de sesión?"

Respuesta: "Se utiliza el guard funcional `sessionGuard` configurado en `canActivate: [sessionGuard]`. Si el usuario no tiene una sesión activa válida en `AuthService`, el guard intercepta la navegación y redirige automáticamente a `/login`."

3. Interpolación {{ }} (Mínimo 3)

ANGULAR TEMPLATES / FRONTEND

¿Qué hace y qué función cumple?

Inserta valores calculados o propiedades dinámicas de la clase TypeScript dentro del código HTML para su visualización en el navegador.

¿Dónde está en el código?

`frontend/src/app/pages/tutores-page/tutores-page.html` :

- Línea 163: `<strong>{{ p.totalElements }} tutores disponibles</strong>`
- Línea 199: `{{ t.nombre }}` (Nombre del tutor en la tarjeta)
- Línea 223: `{{ t.biografia || 'Biografía no registrada.' }}` (Biografía dinámica)
- Línea 228: `<strong>${{ t.tarifaPorHora | number }}</strong>` (Tarifa formateada)

4. Property Binding [ ] (Mínimo 3)

ANGULAR TEMPLATES / FRONTEND

¿Qué hace y qué función cumple?

Establece un enlace unidireccional desde el modelo TypeScript hacia propiedades nativas del DOM o *inputs* de componentes hijos.

¿Dónde está en el código?

`frontend/src/app/pages/tutores-page/tutores-page.html` :

- Línea 46: `[disabled]="cargando()"` (Deshabilita botón si la petición HTTP está en curso)
- Línea 54: `[class.myt-pill-chip--active]="categoriaActiva() === chip"` (Clase condicional)
- Línea 79-80: `[tutor]="tutor" [ocupado]="guardando()"` (Pasa objeto y estado al hijo editor)
- Línea 187: `[class.myt-tutor-card--disabled]="!t.disponible"`

5. Event Binding ( ) (Mínimo 3)

ANGULAR TEMPLATES / FRONTEND

¿Qué hace y qué función cumple?

Escucha eventos emitidos por el usuario (como clicks o envíos de formulario) o eventos personalizados emitidos por componentes hijos para invocar métodos del componente padre.

¿Dónde está en el código?

`frontend/src/app/pages/tutores-page/tutores-page.html` :

- Línea 12: `(ngSubmit)="filtrar()"` (Dispara búsqueda al enviar formulario)
- Línea 55: `(click)="seleccionarCategoriaChip(chip)"` (Filtra por categoría de materia)
- Línea 81: `(guardado)="guardar($event)"` (Recibe datos emitidos por el editor hijo)
- Línea 175: `(click)="alternarOrden()"` (Alterna orden por calificación o precio)
- Línea 251: `(click)="seleccionar(t.id)"` (Carga datos para edición)

6. Two-way Binding [(ngModel)] (Mínimo 2)

FORMSMODULE / FRONTEND

¿Qué hace y qué función cumple?

Mantiene sincronizado el valor del control de entrada en la vista con la variable de la clase en ambas direcciones: cambios en la UI actualizan el modelo y viceversa.

¿Dónde está en el código?

`frontend/src/app/pages/tutores-page/tutores-page.html` :

- Línea 18: `[(ngModel)]="busquedaTexto"` (Campo de búsqueda por texto)
- Línea 26: `[(ngModel)]="materiaId"` (Select de materias)
- Línea 43: `[(ngModel)]="tarifaMax"` (Filtro numérico de tarifa)
- Líneas 112, 123, 133: `[(ngModel)]="usuarioId"` , `[(ngModel)]="biografia"` , `[(ngModel)]="tarifa"`

```
<!— tutores-page.html: Barra de filtros y búsqueda —>
<input type="search" name="busquedaTexto" [(ngModel)]="busquedaTexto" (ngModelChange)="filtrar()" />
<select name="materia" [(ngModel)]="materiaId" (ngModelChange)="filtrar()"> … </select>
<input name="tarifaMax" type="number" [(ngModel)]="tarifaMax" />
```

7. Signals (signal, set y update con contador visible)

ANGULAR CORE / FRONTEND

¿Qué hace y qué función cumple?

Utiliza el nuevo modelo reactivo de Angular basado en primitivas reactivas (*Signals*) para rastrear cambios en el estado y notificar a la vista de manera quirúrgica y ultra rápida sin ciclos pesados de Change Detection.

¿Dónde está en el código?

Archivo: `frontend/src/app/pages/tutores-page/tutores-page.ts`

- **signal():** Línea 52: `readonly cambios = signal(0);` (contador de acciones exitosas).
- **set():** Línea 162: `this.cargando.set(true);` y Línea 185: `this.pagina.set(p);`
- **update():** Línea 307 (guardar): `this.cambios.update((n) ⇒ n + 1);`
- **update():** Línea 332 (crear): `this.cambios.update((n) ⇒ n + 1);`
- **update():** Línea 355 (desactivar): `this.cambios.update((n) ⇒ n + 1);`
- **Visualización:** `tutores-page.html:168` muestra ⚡ `Acciones exitosas: {{ cambios() }}`

```
// tutores-page.ts (extracto)
readonly cambios = signal(0); // Declaración con signal()

guardar(datos: TutorEdicion) {
  this.api.editarTutor(tutor.id, datos).subscribe({
    next: () ⇒ {
      this.seleccionado.set(null);       // Modificación con set()
      this.cambios.update((n) ⇒ n + 1); // Modificación con update()
      this.cargar();
    }
  });
}
```

Demostración en vivo ante el docente:

"Muestre la pantalla de tutores. Inicialmente el contador no aparece o está en 0. Al hacer clic en 'Editar', modificar la biografía o tarifa y dar clic en 'Guardar cambios', la respuesta exitosa del servidor ejecuta `this.cambios.update(n ⇒ n + 1)` y la interfaz muestra de inmediato la insignia ⚡ `Acciones exitosas: 1`."

8. Control de Flujo (@if, @switch, @for)

ANGULAR CONTROL FLOW / FRONTEND

¿Qué hace y qué función cumple?

Reemplaza las antiguas directivas estructurales (`*ngIf` , `*ngSwitch` , `*ngFor`) por la nueva sintaxis integrada de control de flujo de Angular que ofrece mejor rendimiento, tipado seguro y código más limpio.

¿Dónde está en el código?

Archivo: `frontend/src/app/pages/tutores-page/tutores-page.html`

- **@if:**
  Línea 62 (`@if (error())`), Línea 76 (`@if (seleccionado(); as tutor)`), Línea 88 (`@if (auth.sesion()?.rol === 'ADMINISTRADOR')`).
- **@switch:**
  Líneas 156-173 (evalúa el estado de carga `@switch (cargando())` mostrando el indicador o la lista).
- **@for:**
  Líneas 28-30 (`@for (m of materias(); track m.id)`), Línea 50 (chips de categoría), Líneas 186-258 (tarjetas de tutores con `track t.id`).

```
<!— tutores-page.html: Evidencia de los 3 bloques aplicados —>
@if (error()) {
  <div class="myt-alert myt-alert--error"><p>{{ error() }}</p></div>
}

@switch (cargando()) {
  @case (true) { <output>Cargando tutores…</output> }
  @case (false) {
    @if (pagina(); as p) { <p>{{ p.totalElements }} tutores disponibles</p> }
  }
}

@for (t of tutoresMostrados(); track t.id) {
  <article class="myt-tutor-card"> … </article>
}
```

9. Comunicación Padre-Hijo (input, output, emit, $event)

ANGULAR COMPONENT COMMUNICATION / FRONTEND

¿Qué hace y qué función cumple?

Implementa el patrón de selección y edición de un registro. El padre (`TutoresPageComponent`) descarga el tutor seleccionado y se lo pasa al hijo mediante `input()` ; el hijo (`TutorEditorComponent`) precarga el formulario, permite la edición y emite el borrador al padre con `output().emit()` ; el padre lo recibe mediante `$event` y ejecuta el PUT hacia el backend.

¿Dónde está en el código?

- **Hijo:** `frontend/src/app/shared/components/organisms/tutor-editor/tutor-editor.ts` (Líneas 55-80)
- **Padre Template:** `frontend/src/app/pages/tutores-page/tutores-page.html` (Líneas 76-85)
- **Padre Clase:** `frontend/src/app/pages/tutores-page/tutores-page.ts` (Líneas 293-312)

```
// 1. COMPONENTE HIJO: tutor-editor.ts
export class TutorEditorComponent {
  readonly tutor = input.required<TutorApi>();      // input(): Recibe del padre
  readonly ocupado = input(false);                  // input()
  readonly guardado = output<TutorEdicion>();       // output(): Comunica al padre
  readonly cancelado = output<void>();

  guardar() {
    if (this.form.valid && !this.ocupado()) {
      this.guardado.emit(this.form.getRawValue());  // emit(): Envía datos modificados
    }
  }
}

<!— 2. TEMPLATE PADRE: tutores-page.html —>
<app-tutor-editor
  [tutor]="tutor"
  [ocupado]="guardando()"
  (guardado)="guardar($event)"                    <!— $event: Recibe el dato emitido —>
  (cancelado)="seleccionado.set(null)"
/>

// 3. CLASE PADRE: tutores-page.ts
guardar(datos: TutorEdicion) {                     // Recibe el $event con los datos
  const tutor = this.seleccionado();
  this.api.editarTutor(tutor.id, datos).subscribe( ... );
}
```

10, 11 y 12. Formularios Reactivos y Validaciones

REACTIVEFORMSMODULE / VALIDATIONS

¿Qué hace y qué función cumple?

Gestiona formularios complejos mediante un modelo de objetos reactivo (`FormGroup` , `FormControl`) con validación sincrónica estricta, bloqueando el envío si los datos son inválidos y mostrando mensajes de error contextuales al usuario.

¿Dónde está en el código?

Archivo: `frontend/src/app/shared/components/organisms/tutor-editor/tutor-editor.ts`

- **ReactiveFormsModule:**
  Línea 6: `imports: [ReactiveFormsModule]`
- **FormGroup y FormControl:**
  Líneas 59-69
- **Validaciones y Mensajes:**
  Líneas 22-42 y 60-67

```
// tutor-editor.ts: Definición del formulario y sus validaciones
readonly form = new FormGroup({
  biografia: new FormControl('', {
    nonNullable: true,
    validators: [Validators.required, Validators.maxLength(2000)], // Validación 1 y 2
  }),
  tarifaPorHora: new FormControl(1, {
    nonNullable: true,
    validators: [Validators.required, Validators.min(1), Validators.max(9999999999.99)], // Validación 3 y
  }),
  disponible: new FormControl(true, { nonNullable: true }),
});
```

Las 4 Validaciones Implementadas con Alertas al Usuario:

- **Validación 1 (Tarifa requerida):**
  Si se borra la tarifa y el campo es tocado: `<p role="alert">La tarifa es obligatoria.</p>`
- **Validación 2 (Rango numérico):**
  Tarifa menor a 1 o mayor al límite: `<p role="alert">La tarifa debe estar entre 1 y 9999999999.99.</p>`
- **Validación 3 (Biografía requerida):**
  Si se deja en blanco: `<p role="alert">La biografía es obligatoria.</p>`
- **Validación 4 (Longitud máxima):**
  Biografía mayor a 2000 caracteres: `<p role="alert">La biografía no puede superar 2000 caracteres.</p>`
- **Bloqueo del botón Guardar:**
  `[disabled]="form.invalid || ocupado()"` impide el envío si existe cualquier error.

13. Pipes de Angular

ANGULAR PIPES / FRONTEND

¿Qué hace y qué función cumple?

Transforma la visualización de datos sin alterar el valor original en la memoria, adaptándolo a estándares regionales de moneda, fecha, números o capitalización de texto.

¿Dónde está en el código?

- `number` : `tutores-page.html:228` : `${{ t.tarifaPorHora | number }}` y `t.calificacion | number: '1.1-1'`
- `currency` : `reservas-page.html:225` : `{{ costoEstimado() | currency: 'COP' : 'symbol-narrow' : '1.0-0' }} COP`
- `date` : `mensajes-page.html:33` : `{{ r.fecha + 'T12:00:00' | date: 'dd MMM' }}`
- `titlecase` : `reservas-page.html:260` : `{{ e.replace('_', ' ') | titlecase }}`

14, 15 y 16. Servicios, Inyección con inject() y HttpClient

CORE ARCHITECTURE / FRONTEND

¿Qué hace y qué función cumple?

Centraliza toda la comunicación con la API Spring Boot en una única clase inyectable (`ApiService`) utilizando la función nativa `inject()` y el cliente HTTP reactivo `HttpClient` con interceptores.

¿Dónde está en el código?

Archivo: `frontend/src/app/core/services/api.service.ts` (Línea 71-73)

- **Inyección:** `private readonly http = inject(HttpClient);`
- **En componentes:** `readonly api = inject(ApiService);` en `tutores-page.ts:40` y `reservas-page.ts:48` .

17, 18, 19 y 20. Las 8 Operaciones HTTP (GET, POST, PUT, DELETE) con subscribe()

RXJS / HTTP CLIENT

La rúbrica exige 2 operaciones de cada verbo HTTP procesando su respuesta mediante `subscribe()`. El proyecto cuenta exactamente con las 8 operaciones conectadas a la base de datos real:

Verbo	Operación en MyT	Método en ApiService	Suscripción en el Componente Padre
GET #1	Listado paginado de tutores	`api.tutores(page, materia, tarifa)`	`tutores-page.ts:184` : `.subscribe(p ⇒ this.pagina.set(p))`
GET #2	Detalle de tutor por ID para edición	`api.tutor(id)`	`tutores-page.ts:287` : `.subscribe({ next: t ⇒ this.seleccionado.set(t) })`
POST #1	Asignar perfil tutor a cuenta (admin)	`api.crearTutor(datos)`	`tutores-page.ts:329` : `.subscribe({ next: () ⇒ this.cambios.update(n⇒n+1) })`
POST #2	Crear reserva de tutoría (estudiante)	`api.crearReserva(datos)`	`reservas-page.ts:230` : `.subscribe({ next: () ⇒ this.cargar() })`
PUT #1	Guardar cambios de perfil de tutor	`api.editarTutor(id, datos)`	`tutores-page.ts:304` : `.subscribe({ next: () ⇒ this.cambios.update(n⇒n+1) })`
PUT #2	Confirmar o cancelar reserva	`api.estadoReserva(id, estado, motivo)`	`reservas-page.ts:249` : `.subscribe({ next: () ⇒ this.cargar() })`
DELETE #1	Desactivación lógica de tutor	`api.desactivarTutor(id)`	`tutores-page.ts:353` : `.subscribe({ next: () ⇒ this.cambios.update(n⇒n+1) })`
DELETE #2	Cancelación y baja de reserva	`api.cancelarReserva(id, motivo)`	`reservas-page.ts:262` : `.subscribe({ next: () ⇒ this.cargar() })`

21. Manejo de Errores HTTP

ERROR HANDLING / UX

¿Qué hace y qué función cumple?

Captura las respuestas de error del servidor mediante `catchError` de RxJS y la función utilitaria `mensajeHttp()`, traduciendo códigos de estado HTTP en mensajes comprensibles y amigables que preservan los datos digitados por el usuario.

¿Dónde está en el código?

- Función: `frontend/src/app/core/services/api.service.ts` (Líneas 63-71)
- Alerta visual: `frontend/src/app/pages/tutores-page/tutores-page.html` (Líneas 62-68)

```
// api.service.ts: Manejador centralizado de errores HTTP
export function mensajeHttp(error: unknown): string {
  if (!(error instanceof HttpErrorResponse)) return 'No fue posible completar la operación.';
  if (error.status === 0 || error.status ≥ 500)
    return 'No hay conexión con el servidor. Conservamos tus datos; reintenta.'; // Error 1: Servidor / Re
  if (error.status === 401) return error.error?.mensaje || 'Tu sesión expiró. Vuelve a ingresar.';
  if (error.status === 403) return error.error?.mensaje || 'Tu perfil no tiene permiso para esta acción.';
  if (error.status === 409)
    return error.error?.mensaje || 'Conflicto: el registro ya existe o cambió. Revisa y reintenta.'; // Er
  return error.error?.mensaje || 'Revisa los datos ingresados y el horario disponible.';
}
```

22. Loading (Indicación Visual de Espera)

ANGULAR UX / SIGNALS

¿Qué hace y qué función cumple?

Brinda feedback visual inmediato al usuario mientras una petición HTTP se encuentra en tránsito en la red, evitando peticiones duplicadas y deshabilitando controles.

¿Dónde está en el código?

• Señales: `readonly cargando = signal(false);` y `readonly guardando = signal(false);` en `tutores-page.ts:49-50` .
• Control con RxJS: `finalize(() => this.cargando.set(false))` en `tutores-page.ts:179` .
• Plantilla: `tutores-page.html:156` (`@switch (cargando()) { @case (true) { <output>Cargando tutores…</output> } }`) y línea 71 (`@if (guardando()) { <output>Procesando cambios…</output> }`).

23 y 24. REST API Spring Boot (@RestController) y ResponseEntity

SPRING BOOT / JAVA 21

¿Qué hace y qué función cumple?

Implementa controladores REST desacoplados que exponen endpoints HTTP con serialización JSON, inyección de dependencias por constructor y encapsulamiento de respuestas en `ResponseEntity` con códigos de estado semánticos apropiados (200, 201, 204, 400, 404).

¿Dónde está en el código?

`backend/src/main/java/com/typeerror/myt/api/TutorRestController.java`
`backend/src/main/java/com/typeerror/myt/api/ReservaRestController.java`

```
// TutorRestController.java (extracto de endpoints y códigos ResponseEntity)
@RestController
@RequestMapping("/api/tutores")
public class TutorRestController {

  // 1. GET con 200 OK y 400 Bad Request
  @GetMapping
  public ResponseEntity<Page<ApiDtos.TutorDto>> listar(@RequestParam(defaultValue = "0") int page, ...) {
    if (page < 0 || size < 1) return ResponseEntity.badRequest().build(); // 400 ERROR
    return ResponseEntity.ok(pagina.map(...));                            // 200 ÉXITO
  }

  // 2. GET por ID con 200 OK y 404 Not Found
  @GetMapping("/{id}")
  public ResponseEntity<ApiDtos.TutorDto> consultar(@PathVariable Integer id) {
    return tutores.findById(id).map(t -> ResponseEntity.ok(dto(t)))       // 200 ÉXITO
      .orElseGet(() -> ResponseEntity.notFound().build());                // 404 ERROR
  }

  // 3. POST con 201 Created y URI del nuevo recurso
  @PostMapping
  public ResponseEntity<ApiDtos.TutorDto> crear(@Valid @RequestBody ApiDtos.TutorAlta datos, ...) {
    Tutor tutor = tutores.save(...);
    return ResponseEntity.created(URI.create("/api/tutores/" + tutor.getId())) // 201 ÉXITO
      .body(ApiDtos.tutor(tutor, null));
  }

  // 4. PUT con 200 OK y 404 Not Found
  @PutMapping("/{id}")
  public ResponseEntity<ApiDtos.TutorDto> editar(@PathVariable Integer id, ...) {
    return tutores.findById(id).map(t -> {
      return ResponseEntity.ok(dto(tutores.save(t)));                     // 200 ÉXITO
    }).orElseGet(() -> ResponseEntity.notFound().build());                // 404 ERROR
  }

  // 5. DELETE con 204 No Content
  @DeleteMapping("/{id}")
  public ResponseEntity<Void> desactivar(@PathVariable Integer id, ...) {
    tutor.get().setDisponible(false);
    tutores.save(tutor.get());
    return ResponseEntity.noContent().build();                            // 204 ÉXITO
  }
}
```

25. Paginación en Backend y Frontend

FULLSTACK / PAGINATION

¿Qué hace y qué función cumple?

Divide conjuntos voluminosos de datos en bloques discretos (páginas), optimizando la transferencia de red y la memoria del cliente. El backend utiliza `PageRequest.of(page, size)` y el frontend renderiza controles de navegación.

¿Dónde está en el código?

- **Backend:** `TutorRestController.java:69-75` (Crea `PageImpl` y retorna `Page<TutorDto>`).
- **Frontend:** `tutores-page.ts:107-127` (Calcula `paginasDisponibles`) y métodos `irAPagina()` y `navegar()`.
- **Datos:** El script de seed incluye 30 tutores que generan 6 páginas navegables de 5 elementos cada una.

26. Conexión PostgreSQL y Persistencia Real

DATABASE / POSTGRESQL

¿Qué hace y qué función cumple?

Garantiza la persistencia relacional estricta bajo estándares ACID. No utiliza datos ficticios en memoria ni mocks: los cambios persisten entre reinicios y recargas de página.

¿Dónde está en el código?

- **Configuración:** `backend/src/main/resources/application.properties` y perfil `application-dev.properties`.
- **Esquema:** Esquema dedicado `app` para aislamiento de datos.
- **Migraciones:** `backend/supabase/migrations/` (archivos 001 a 004 aplicados cronológicamente).

27. JPA / Hibernate (Entidades y Relaciones)

ORM / SPRING DATA JPA

¿Qué hace y qué función cumple?

Mapea las tablas de PostgreSQL a clases Java mediante JPA/Hibernate, gestionando ciclos de vida transaccionales, restricciones de integridad referencial y carga optimizada con `@EntityGraph`.

¿Dónde está en el código?

Paquete `backend/src/main/java/com/typeerror/myt/entities/`:

- `Usuario.java`: `@Entity`, `@Table(name="usuarios", schema="app")`.
- `Tutor.java`: `@OneToOne` con Usuario, `@ManyToMany` con Materia (tabla intermedia `tutor_materias`), `@OneToMany` con Disponibilidad y Reserva.
- `Reserva.java`: `@ManyToOne` con Estudiante, Tutor, Materia y `@OneToOne` con Resena.

28. Consultas JPQL (Mínimo 3 Consultas Diferentes)

JPQL / SPRING DATA JPA

El proyecto cuenta con 3 consultas explícitas escritas en **Java Persistence Query Language (JPQL)** mediante la anotación `@Query`:

```
// CONSULTA JPQL 1: Promedio de calificación con agregación AVG y múltiples JOIN
// Archivo: backend/src/main/java/com/typeerror/myt/repository/TutorRepository.java (líneas 49-57)
@Query("""
    SELECT AVG(resena.calificacion)
    FROM Tutor tutor
    JOIN tutor.reservas reserva
    JOIN reserva.resena resena
    WHERE tutor.id = :tutorId
    """)
Optional<Double> findCalificacionPromedioByTutorId(@Param("tutorId") Integer tutorId);

// CONSULTA JPQL 2: Búsqueda dinámica con parámetros opcionales y paginación
// Archivo: backend/src/main/java/com/typeerror/myt/repository/TutorRepository.java (líneas 69-80)
@EntityGraph(attributePaths = {"usuario", "usuario.roles", "materias"})
@Query("""
    SELECT DISTINCT t FROM Tutor t LEFT JOIN t.materias m
    WHERE (:materiaId IS NULL OR m.id = :materiaId)
    AND (:tarifaMax IS NULL OR t.tarifaPorHora ≤ :tarifaMax)
    AND t.usuario.activo = true
    AND t.disponible = true
    """)
Page<Tutor> buscar(@Param("materiaId") Integer materiaId,
                   @Param("tarifaMax") BigDecimal tarifaMax,
                   Pageable pageable);

// CONSULTA JPQL 3: Historial de reservas de estudiante filtradas por estado y ordenadas
// Archivo: backend/src/main/java/com/typeerror/myt/repository/ReservaRepository.java (líneas 43-52)
@EntityGraph(attributePaths = {"estudiante", "estudiante.usuario", "tutor", "tutor.usuario", "materia", "r
@Query("""
    SELECT r FROM Reserva r WHERE r.estudiante.id = :estudianteId
    AND (:estado IS NULL OR r.estado = :estado)
    ORDER BY r.fecha DESC, r.horaInicio DESC, r.id DESC
    """)
List<Reserva> buscarPorEstudiante(@Param("estudianteId") Integer estudianteId,
                                  @Param("estado") EstadoReserva estado);
```

4. Guion Paso a Paso para la Sustentación en Vivo (5 a 10 min)

Sigue este recorrido ordenado frente al docente para demostrar el 100% de los elementos solicitados:

Paso 1: Demostrar Catálogo, Routing, Pipes e Interpolación (2 minutos)

1. Inicia sesión como Administrador (`admin.ortiz@myt.edu.co` / `1ManzanaGrande!`) o Tutor.
2. Navega a `http://localhost:4200/tutores` .
3. Muestra el **Routing** en la barra de direcciones y explica cómo `provideRouter` en `app.routes.ts` gestiona la navegación SPA sin recargar.
4. Señala la **Interpolación {{ }}**: `{{ p.totalElements }}` (muestra "30 tutores disponibles") y `{{ t.nombre }}` .
5. Señala los **Pipes**: Tarifa `${{ t.tarifaPorHora | number }}` y `{{ t.calificacion | number: '1.1-1' }}` .
6. Señala la **Paginación**: Navega entre las páginas 1 a 6 (demostrando que se solicitan bloques de 5 tutores mediante `page=0, size=5`).

Paso 2: Demostrar Two-Way Binding y Property/Event Binding (1 minuto)

1. En la barra de búsqueda, escribe "Cálculo". Muestra cómo `[(ngModel)]="busquedaTexto"` sincroniza la búsqueda instantáneamente con `(ngModelChange)="filtrar()"` .
2. Abre la herramienta de desarrollo (F12) en la pestaña **Network**: observa la petición GET a `/api/tutores?page=0&size=5&q=Calculo` .

Paso 3: Demostrar Padre-Hijo, Formularios Reactivos y Validaciones (2.5 minutos)

1. Haz clic en el botón **"Editar"** en una tarjeta de tutor (Event Binding `(click)="seleccionar(t.id)"`).
2. Explica al profesor:
   - El padre descarga el tutor con `api.tutor(id)` y se lo pasa al hijo mediante `[tutor]="tutor"` (**`input()`**).
   - El componente hijo `TutorEditorComponent` precarga el **`FormGroup`** con **`FormControl`** (Formulario Reactivo).
3. **Demostrar Validaciones:**
   - Borra la tarifa: aparece la alerta roja *"La tarifa es obligatoria."* y el botón *Guardar cambios* queda deshabilitado.
   - Coloca tarifa 0: aparece *"La tarifa debe estar entre 1 y 9999999999.99."*.
   - Borra la biografía: aparece *"La biografía es obligatoria."*.
4. Modifica la tarifa a **55000** y haz clic en **Guardar cambios**:
   - El hijo ejecuta `this.guardado.emit(datos)` (**`output()`** y **`emit()`**).
   - El padre escucha `(guardado)="guardar($event)"` y envía el PUT a `/api/tutores/{id}` .

Paso 4: Demostrar Signals (signal, set, update) y Loading (1.5 minutos)

1. Inmediatamente después de guardar, señala en la interfaz la insignia: ⚡ `Acciones exitosas: 1` (producida por `this.cambios.update(n ⇒ n + 1)`).
2. Explica: "Utilizamos `cambios = signal(0)` ; cada operación exitosa incrementa el contador con `update()` y el estado se actualiza en la vista automáticamente."
3. En Network, pon la velocidad en **Slow 3G**: haz clic en 'Filtrar' y muestra cómo aparece el mensaje `Cargando tutores…` gracias al bloque `@switch (cargando())` .

Paso 5: Demostrar las 8 Operaciones HTTP, ResponseEntity y Persistencia (2 minutos)

1. **GET #1 y #2:** Ya mostrados al consultar el catálogo y abrir el editor.

2. **PUT #1:** Ya demostrado al guardar la edición del tutor (retorna `200 OK`).
3. **POST #1:** Despliega el panel "+ *Asignar perfil tutor*", ingresa un ID de estudiante y una tarifa, y da clic en Crear (retorna `201 Created`).
4. **DELETE #1:** Haz clic en "*Desactivar tutor*" y confirma: el estado cambia a `disponible=false` en PostgreSQL (retorna `204 No Content`).
5. **Persistencia real:** Recarga el navegador con **Ctrl + F5**: comprueba que los datos persisten exactamente como fueron modificados.
6. **Manejo de Errores:** Desconecta la red (Modo Offline en DevTools) e intenta filtrar: la aplicación captura el status 0 y muestra: "*No hay conexión con el servidor. Conservamos tus datos; reintenta.*".

Paso 6: Mostrar el Backend y las Consultas JPQL (1 minuto)

1. Abre en tu IDE (VS Code o IntelliJ) el archivo `TutorRepository.java`: muestra la consulta 1 con `AVG(resena.calificacion)` y la consulta 2 con el filtro dinámico `WHERE (:materiaId IS NULL OR ...)`.
2. Abre `ReservaRepository.java` y muestra la consulta 3: `buscarPorEstudiante` con filtro opcional de estado.
3. Abre `TutorRestController.java` y muestra el uso de `ResponseEntity.ok()`, `ResponseEntity.created()`, `ResponseEntity.noContent()`, `ResponseEntity.badRequest()` y `ResponseEntity.notFound()`.

5. Comandos de Arranque y Credenciales de Demostración

5.1 Credenciales de Acceso

Perfil	Correo Electrónico	Contraseña Universal	Funcionalidades Clave
Administrador	`admin.ortiz@myt.edu.co`	`1ManzanaGrande!`	Asignar perfil tutor, listar usuarios, ver todas las tutorías.
Estudiante	`estudiante01@myt.edu.co`	`1ManzanaGrande!`	Explorar tutores, reservar horarios, cancelar tutorías propias.
Tutor	`tutor01@myt.edu.co`	`1ManzanaGrande!`	Editar tarifa y biografía, confirmar o cancelar reservas.

5.2 Comandos de Ejecución en PowerShell

```
# 1. Iniciar Base de Datos PostgreSQL local en Docker y Backend Spring Boot
cd "UNI-26-30-Web-Proyecto"
./backend/run-demo.ps1

# 2. En otra terminal PowerShell, iniciar Frontend Angular
cd "UNI-26-30-Web-Proyecto/frontend"
npm ci
npm start

# La aplicación queda disponible en: http://localhost:4200
```

5.3 Comprobación Rápida de Pruebas Unitarias

```
# Pruebas Backend (122 pruebas pasando)
cd backend
./mvnw.cmd test

# Pruebas Frontend (96 pruebas pasando)
cd frontend
npm run test:ci
```

¡Preparado para la sustentación individual!

Este documento cubre el 100% de los puntos de la rúbrica con trazabilidad de código demostrable en vivo.